Глава 2. Стратегии моделирования угроз

Чем раньше вы находите проблемы, тем легче их исправить. Моделирование угроз — это всё о поиске проблем, и поэтому его следует делать рано в процессе разработки или проектирования, или при подготовке к развёртыванию операционной системы. Есть много способов моделировать угрозы. Некоторые способы очень специфичны, как набор для сборки модели самолёта, который можно использовать только для построения истребителя F-14. Другие методы более универсальны, как строительные блоки Lego, из которых можно сделать множество вещей. Некоторые методы моделирования угроз плохо сочетаются, как детали конструктора Erector и блоки Lego. Эта глава охватывает различные стратегии и методы, которые применялись к моделированию угроз, представляет каждый из них глубоко и закладывает основу для эффективного поиска угроз.

Вы начнёте с очень простых методов, таких как вопрос «какова ваша модель угроз?» и мозговой штурм об угрозах. Они могут работать для эксперта по безопасности и могут сработать для вас. Оттуда вы узнаете о трёх стратегиях моделирования угроз: фокус на активах, фокус на атакующих и фокус на ПО. Эти стратегии более структурированы и могут работать для людей с разными навыками. Фокус на ПО обычно является наиболее подходящей стратегией. Желание сосредоточиться на активах или атакующих естественно и часто представляется как неизбежный или существенный аспект моделирования угроз. Было бы неправильно не представить каждый из них в лучшем свете, прежде чем обсуждать проблемы с этими стратегиями. Оттуда вы узнаете о различных типах диаграмм, которые можно использовать для моделирования вашей системы или ПО.

ПРИМЕЧАНИЕ. Эта глава не включает конкретные строительные блоки для обнаружения угроз, которые являются предметом следующих нескольких глав.

«Какова ваша модель угроз?»

Вопрос «какова ваша модель угроз?» — отличный, потому что всего четырьмя словами он может разрубить многие загадки, чтобы определить, о чём вы беспокоитесь. Ответы часто имеют форму «атакующий с ноутбуком» или «инсайдеры», или (к сожалению, часто) «а?» Ответ «а?» полезен, потому что он показывает, сколько работы потребуется, чтобы найти последовательный и структурированный подход к защите. Последовательность и структура важны, потому что они помогают вам инвестировать в защиты, которые остановят атакующих. Существует сборник стандартных ответов на вопрос «какова ваша модель угроз?» в Приложении А «Helpful Tools», но несколько примеров приведены и здесь:

- Вор, который мог бы украсть ваши деньги
- Заинтересованные стороны компании (сотрудники, консультанты, акционеры и т.д.), которые имеют доступ к чувствительным документам и не пользуются доверием
- Недоверенная сеть
- Атакующий, который мог бы украсть ваш cookie (веб или иной)

ПРИМЕЧАНИЕ. На протяжении этой книги вы будете возвращаться к одному и тому же примеру для каждого из этих подходов. Ваши основные цели — вымышленный коммерческий сервер базы данных «Acme/SQL» корпорации Acme и операционная сеть Acme. Используя примеры Acme, вы увидите, как различные подходы разворачиваются против одних и тех же систем.

Применяя вопрос «какова ваша модель угроз?» к примеру корпорации Acme, вы можете получить следующие ответы:

Для базы данных Acme SQL модель угроз — это атакующий, который хочет читать или изменять данные в базе данных. Более тонкая модель может также включать людей, которые хотят читать данные, не появляясь в логах. Для финансовой системы Acme ответы могут включать кого-то, получающего чек, которого он не заслуживает, клиентов, которые не вносят причитающийся платёж, и/или кого-то, читающего или изменяющего финансовые результаты до их публикации.

Если у вас нет чёткого ответа на вопрос «какова ваша модель угроз?», это может привести к непоследовательности и потраченным впустую усилиям. Например, у стартапа Zero-Knowledge Systems не было чёткого ответа на вопрос «какова ваша модель угроз?» Поскольку чёткого ответа не было, не было и последовательности в том, какие функции безопасности строились. Огромное количество энергии ушло на построение защит против самых сложных атак, и эти решения по защите от таких атак влияли на производительность всей системы. Хотя предотвращение слежки правительств за клиентами было весёлой технической задачей и эмоционально резонирующей целью, и задача, и эмоциональное воздействие затрудняли принятие технических решений, которые могли бы сделать бизнес более успешным. В конце концов, более чёткий ответ на вопрос «какова ваша модель угроз?» позволил Zero-Knowledge Systems инвестировать в меры, которые все адресовали один и тот же поднабор возможных угроз.

Так как же убедиться, что у вас есть чёткий ответ на этот вопрос? Часто ответы не очевидны даже для тех, кто регулярно думает о безопасности, и сам вопрос предлагает мало структуры для выяснения ответов. Один из подходов, часто рекомендуемый, — это мозговой штурм. В следующем разделе вы узнаете о различных подходах к мозговому штурму и связанных с ними компромиссах.

Мозговой штурм ваших угроз

Мозговой штурм — самый традиционный способ перечисления угроз. Вы собираете группу опытных экспертов в комнате, даёте им способ делать заметки (доски или коктейльные салфетки традиционны) и позволяете им работать. Качество мозгового штурма ограничено опытом участников и количеством времени, потраченным на него.

Мозговой штурм включает период генерации идей, за которым следует период анализа и отбора идей. Мозговой штурм для моделирования угроз включает придумывание возможных атак всех видов. Во время фазы генерации идей следует запретить критику. Вы хотите исследовать пространство возможных угроз, и атмосфера критики будет подавлять такую генерацию идей. Модератор может помочь поддерживать движение мозгового штурма.

Во время мозгового штурма ключевым является наличие в комнате эксперта по технологии, которая моделируется. Иначе легко сделать неверные предположения о том, как она работает. Однако, когда у вас есть эксперт, гордящийся своей технологией, вам нужно убедиться, что вы не получите «гордого родителя», оскорблённого тем, что его программное дитя называют уродливым. Полезное правило: атакуется ПО, а не архитекторы ПО. Это не всегда достаточно, но это хорошее начало. Также есть польза в объединении разнообразной группы экспертов с более широким набором опыта.

Мозговой штурм также может выродиться в атаки вне области охвата. Например, если вы проектируете программу для чата, атаки блока управления памятью против CPU, вероятно, вне области охвата, но если вы проектируете материнскую плату, эти атаки могут быть фокусом вашего моделирования угроз. Один из способов справиться с этой проблемой — составить список атак, которые вне области охвата, таких как «администратор злонамерен» или «атакующий редактирует жёсткий диск на другой системе», а также набор эквивалентностей атак, например «атакующий может разрушить стек и выполнить код», чтобы эти проблемы можно было признать и обрабатывать последовательно. Вариант мозгового штурма — призыв «думать как атакующий», который более подробно обсуждается в главе 18 «Experimental Approaches».

Некоторые атаки, которые вы могли бы придумать при моделировании угроз для финансовой отчётности Acme, включают взлом через Интернет, спаивание CFO, подкуп уборщика или предсказание URL, где будут опубликованы финансовые отчёты. Это может быть сборной солянкой, поэтому следующий раздел предлагает несколько более сфокусированные подходы.

Варианты мозгового штурма

Свободный или «нормальный» мозговой штурм, как обсуждалось в предыдущем разделе, может использоваться как метод моделирования угроз, но есть более конкретные методы, которые можно использовать, чтобы помочь сфокусировать ваш мозговой штурм. В следующих разделах описываются variations классического мозгового штурма: анализ сценариев, пре-мортемы и киносюжеты.

Анализ сценариев

Может помочь сфокусировать ваш мозговой штурм с помощью сценариев. Если вы используете письменные сценарии в вашей общей инженерии, вы можете начать с них и спросить, что может пойти не так, или вы можете использовать вариант закона Чандлера («Когда сомневаешься, пусть в дверь войдёт человек с пистолетом в руке»). Вам не нужно ограничиваться человеком с пистолетом, конечно; вы можете использовать любого из атакующих, перечисленных в Приложении C «Attacker Lists».

Для примера сценарий-специфичного мозгового штурма попробуйте смоделировать угрозы для ситуации, когда вы передаёте свой телефон симпатичному человеку в баре. Это интересное упражнение. Получатель мог бы, например, отправить пожертвования Красному Кресту, написать важному человеку «перестань меня доставать» или опубликовать в Facebook, что «я не понимаю намёков» или «я сомнительный тип», не говоря уже о возможности убежать с телефоном или уронить его в пиво.

Менее frivolous, ваши примерные сценарии могут быть основаны на продуктовых сценариях или вариантах использования для текущего цикла разработки и, следовательно, охватывать отказоустойчивость и репликацию, и как эти сервисы могут быть эксплуатированы, когда они не правильно аутентифицированы и авторизованы.

Пре-мортем

Эксперт по наукам о решениях Гэри Кляйн предложил другую технику мозгового штурма, которую он называет пре-мортем (Klein, 1999). Идея состоит в том, чтобы собрать участников решения и попросить их предположить, что вскоре после крайнего срока проекта или после ключевой вехи дела пошли совсем не так. С «предположением о провале» в уме идея состоит в том, чтобы исследовать, почему участники считают, что всё пойдёт не так. Ценность называния этого пре-мортемом — в рамке, которую это создаёт. Естественный оптимизм, сопровождающий проект, заменяется явным предположением, что он провалился, давая вам и другим участникам шанс выразить сомнения. В моделировании угроз предположение состоит в том, что продукт успешно атакуется, и теперь у вас есть разрешение выразить сомнения или опасения.

Киносюжеты

Ещё один вариант мозгового штурма — киносюжеты. Ключевое отличие «нормального мозгового штурма» от «киносюжетов» в том, что идеи атак предназначены быть возмутительными и провокационными, чтобы поощрить свободный поток идей. Защита от этих угроз, вероятно, включает научно-фантастические устройства, которые посягают на человеческое достоинство, свободу и приватность, фактически никого не защищая. Примеры отличных фильмов для киносюжетных угроз включают «Одиннадцать друзей Оушена», «Ограбление по-итальянски» и каждый фильм о Бонде, который не отстой. Если вы хотите заняться более структурированным киносюжетным моделированием, создайте три списка: несовершенные протагонисты, блестящие антагонисты и навороченные гаджеты. Затем вы можете комбинировать их по своему усмотрению.

Примеры киносюжетных угроз включают иностранного шпиона, пишущего код для Acme SQL так, что четвёртая попытка подключения позволяет кому-то войти как admin; коварного CFO, крадущего у фирмы; и кого-то, спускающегося с потолка, чтобы избежать ковриков давления на полу, одновременно взламывая базу данных с консоли. Обратите внимание, что эти киносюжеты одинаково применимы к Acme и её клиентам.

Термин «киносюжеты» был придуман Брюсом Шнайером, уважаемым экспертом по безопасности. Объявляя свой конкурс на выявление киносюжетных угроз, он сказал: «Цель этого конкурса — абсурдный юмор, но я надеюсь, что он также докажет одну мысль. Терроризм — реальная угроза, но мы не становимся безопаснее благодаря мерам безопасности, которые требуют от нас правильно угадывать, что террористы сделают дальше» (Schneier, 2006). Это не относится только к терроризму; запутанные, но ярко описанные угрозы могут быть угрозой для вашей методологии моделирования угроз.

Обзор литературы

Как предшественник мозгового штурма (или любого другого подхода к поиску угроз), обзор угроз для систем, похожих на вашу, является полезной отправной точкой в моделировании угроз. Вы можете сделать это с помощью поисковых систем или проверив

академическую литературу и перейдя по ссылкам. Может быть невероятно полезно искать по конкурентам или связанным продуктам. Для начала поищите по конкуренту, добавив термины, такие как «security», «security bug», «penetration test», «pwning» или «Black Hat», и используйте свою креативность. Вы также можете просмотреть общие угрозы в этой книге, особенно часть III «Managing and Addressing Threats» и приложения. Кроме того, «Security Engineering» Росса Андерсона — отличная коллекция реальных мировых атак и инженерных уроков, на которые вы можете опереться, особенно если то, что вы создаёте, похоже на то, что он охватывает (Wiley, 2008).

Обзор литературы по угрозам для баз данных может привести к пониманию атак SQL-инъекций, сбоев резервного копирования, инсайдерских атак, предполагающих необходимость логов. Проведение обзора особенно полезно для тех, кто развивает свои навыки в моделировании угроз. Имейте в виду, что многие угрозы, которые могут возникнуть, могут быть сверхспецифичными. Относитесь к ним как к примерам более общих случаев и ищите variations и связанные проблемы по мере мозгового штурма.

Перспектива на мозговой штурм

Мозговой штурм и его варианты страдают от множества проблем. Мозговой штурм часто порождает угрозы, которые трудно или невозможно устранить. Мозговой штурм по своей сути требует удаления ограничений или границ, и угрозы очень зависят от участников и того, как проходит сессия. Когда эксперты собираются вместе, часто возникает неструктурированное обсуждение. Это может быть весело для экспертов, и обычно даёт интересные результаты, но часто эксперты в дефиците. В других случаях инженеры расстраиваются из-за непоследовательности: «спроси двух экспертов — получишь три ответа».

Есть ещё одна проблема, которую следует рассмотреть, и она связана с критериями выхода. Трудно понять, когда вы закончили мозговой штурм, и закончили ли вы потому, что сделали хорошую работу, или потому, что все просто устали. Руководство инженерного отдела может потребовать оценку времени, которую они могут вставить в своё расписание, а такие оценки трудно предсказать. Лучший подход, чтобы избежать этой проблемы с оценкой времени, — просто назначить встречу определённой длины. К сожалению, этот вариант не даёт высокой уверенности в том, что все интересные угрозы были найдены.

Из-за трудности устранения угроз, выявленных с помощью безграничной техники мозгового штурма, и плохо определённых критериев выхода для сессии мозгового штурма, важно рассмотреть другие подходы к моделированию угроз, которые более предписывающие, формальные, воспроизводимые или менее зависимые от способностей и знаний участников. Такие подходы являются предметом остальной части этой главы, а также обсуждаются в остальной части части II «Finding Threats».

Структурированные подходы к моделированию угроз

Когда трудно ответить на вопрос «какова ваша модель угроз?», люди часто используют подход, сосредоточенный на моделях своих активов, моделях атакующих или моделях своего ПО. Центрирование на одном из них предпочтительнее использования подходов,

которые пытаются их комбинировать, потому что такие комбинации, как правило, запутанны.

Активы — это ценные вещи, которые у вас есть. Люди, которые могут покуситься на ваши активы, — это атакующие, и наиболее распространённый способ для них атаковать — через ПО, которое вы создаёте или развёртываете.

Каждый из этих подходов — естественное место для начала размышлений об угрозах, и у каждого есть преимущества и недостатки, которые рассматриваются в этом разделе. Есть люди с очень сильными мнениями, что один из них правильный (или неправильный). Не беспокойтесь о «правильном» или «неправильном», а о «полезности». То есть, помогает ли ваш подход находить проблемы? Если нет, он неправильный для вас, как бы настойчиво кто-то ни аргументировал его достоинства.

Эти три подхода можно рассматривать как аналогичные наборам Lincoln Logs, Erector и Lego. У каждого есть множество деталей, и каждый позволяет вам строить вещи, но они могут не комбинироваться так произвольно, как вам хотелось бы. То есть вы не можете прикрепить блоки Lego к модели Erector. Аналогично, вы не всегда можете прикрепить атакующих к модели ПО и получить что-то, что работает как связное целое.

Чтобы понять эти три подхода, может быть полезно применить их к чему-то конкретному. На рисунке 2-1 показана диаграмма потоков данных системы Acme/SQL.

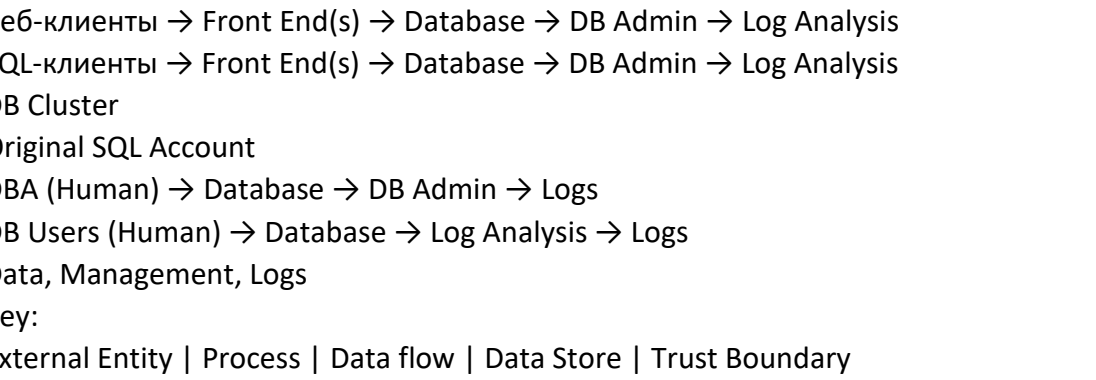


Рисунок 2-1: Диаграмма потоков данных базы данных Acme/SQL

Глядя на диаграмму и читая слева направо, вы видите два типа клиентов, обращающихся к фронтендам и ядру базы данных, которое управляет транзакциями, контролем доступа, атомарностью и так далее. Здесь предположим, что система Acme/SQL поставляется со встроенным веб-сервером, и авторизованным клиентам предоставляется почти необработанный доступ к данным. Одновременно могли бы быть веб-серверы, предлагающие более глубокую бизнес-логику, доступ к нескольким бэкендам, интеграцию с платёжными системами и так далее. Эти веб-серверы обращались бы к Acme/SQL через протокол SQL по сетевому соединению.

Возвращаясь к диаграмме, рисунок 2-1 также показывает набор инструментов DB Admin, которые DBA (человек-администратор базы данных) использует для управления системой. Как показано на диаграмме, есть три концептуальных хранилища данных: Data, Management (включая метаданные, такие как блокировки, политики и индексы) и Logs. Они могут быть реализованы в памяти, как файлы, как пользовательские хранилища на raw-диске или делегированы сетевому хранилищу. По мере углубления детали имеют

большое значение, но обычно вы начинаете моделирование с концептуального уровня, как показано.

Наконец, есть пакет анализа логов. Обратите внимание, что только ядро базы данных имеет прямой доступ к данным и управляющей информации в этом дизайне. Вы также должны заметить, что большинство стрелок двунаправленные, кроме Database ↔ Logs и Logs ↔ Log Analysis. Конечно, процесс Log Analysis будет запрашивать логи, но поскольку он предназначен как интерфейс только для чтения, он представлен как однонаправленная стрелка. Очень редко у вас могут быть строго однонаправленные потоки, такие как реализованные SNMP-ловушками или syslog. Некоторые специалисты по моделированию угроз предпочитают две однонаправленные стрелки, которые могут помочь увидеть угрозы в каждом направлении, но также приводят к перегруженным диаграммам, которые трудно подписывать или читать. Если ваши диаграммы просты, пара однонаправленных стрелок помогает находить угрозы и, следовательно, лучше, чем двунаправленные. Если ваша диаграмма сложна, можно использовать любой подход.

Диаграмма потоков данных более подробно обсуждается позже в этой главе, в разделе «Data Flow Diagrams». В следующих нескольких разделах вы увидите, как применять модели, ориентированные на активы, атакующих и ПО, для поиска угроз против Acme/SQL.

Фокус на активах

Кажется очень естественным центрировать ваш подход на активах, или вещах, представляющих ценность. В конце концов, если вещь не имеет ценности, зачем беспокоиться о том, как кто-то может её атаковать? Оказывается, фокус на активах менее полезен, чем вы можете надеяться, и поэтому не является лучшим подходом к моделированию угроз. Однако есть небольшое число людей, которым принесёт пользу моделирование угроз, ориентированное на активы. Наиболее вероятно получат пользу команда экспертов по безопасности с опытом структурирования своего мышления вокруг активов. (Найдя способ, который работает для них, может не быть причин меняться.) Менее технические люди могут внести вклад в моделирование угроз, сказав «сосредоточьтесь на этом активе». Если вы в одной из этих групп или работаете с ними, поздравляю! Этот раздел для вас. Если вы не один из этих людей, однако, не спешите пропускать вперёд. Всё ещё важно иметь хорошее понимание роли, которую активы могут играть в моделировании угроз, даже если они не в главной роли. Это также может помочь вам понять, почему подход не так полезен, как может показаться, чтобы вы могли вести информированное обсуждение с теми, кто его продвигает.

Термин «актив» обычно относится к чему-то ценному. Когда люди поднимают активы как часть деятельности по моделированию угроз, они часто имеют в виду что-то, к чему атакующий хочет получить доступ, что хочет контролировать или уничтожить. Если у всех, кто участвует в процессе моделирования угроз, нет рабочего соглашения о том, что такое актив, ваш процесс либо застопорится, либо участники будут просто говорить мимо друг друга.

Есть три способа, которыми термин «актив» обычно используется в моделировании угроз:

- Вещи, которые хотят атакующие

- Вещи, которые вы хотите защитить
- Ступеньки к любому из них

Вы должны думать об этих трёх типах активов как о семействах, а не категориях, потому что, как люди могут принадлежать более чем к одной семье одновременно, активы могут принимать более чем одно значение одновременно. Другими словами, теги, применимые к активам, могут пересекаться, как показано на рисунке 2-2. Наиболее распространённое использование термина «актив» в обсуждении моделей угроз, по-видимому, — это сочетание «вещей, которые хотят атакующие» и «вещей, которые вы хотите защитить».

Вещи, которые вы защищаете
Ступеньки
Вещи, которые хотят атакующие

Рисунок 2-2: Пересекающиеся определения активов

ПРИМЕЧАНИЕ. Есть несколько других способов, которыми термин «актив» используется теми, кто моделирует угрозы, — например, как синоним компьютера или типа компьютера (например, «Целевые активы: почтовый сервер, база данных»). Для ясности эта книга использует «актив» только с явной ссылкой на одно или несколько из трёх ранее определённых семейств, и вам следует стараться делать то же самое.

Вещи, которые хотят атакующие

Обычно активы, которые хотят атакующие, — это относительно осязаемые вещи, такие как «эта база данных медицинских данных клиентов». Хорошие примеры вещей, которые хотят атакующие, включают:

- Пароли или ключи пользователей
- Номера социального страхования или другие идентификаторы
- Номера кредитных карт
- Ваши конфиденциальные деловые данные

Вещи, которые вы хотите защитить

Есть также семейство активов, которые вы хотите защитить. В отличие от осязаемых вещей, которые хотят атакующие, многие из этих активов нематериальны. Например, репутация или гудвилл вашей компании — это то, что вы хотите защитить. Конкуренты или активисты вполне могут атаковать вашу репутацию, занимаясь диффамацией. С точки зрения моделирования угроз, ваша репутация обычно слишком диффузна, чтобы можно было технологически смягчить угрозы против неё. Поэтому вы хотите защитить её, защищая вещи, которые важны для ваших клиентов.

Как пример чего-то, что вы хотите защитить: если у вас был пустой сейф, вы интуитивно не хотите, чтобы кто-то пришёл и приложил к нему стетоскоп. Но в нём ничего нет, так какой ущерб? Смена комбинации и сообщение правильным людям (и только правильным людям) новой комбинации требует работы. Поэтому вы хотите защитить этот пустой сейф, но он был бы маловероятной целью для вора. Если в том же сейфе миллион долларов, он

гораздо более вероятно вызовет интерес вора. Миллион долларов — часть семейства вещей, которые вы хотите и которые хотят атакующие.

Ступеньки

Последнее семейство активов — это ступеньки к другим активам. Например, всё, что нарисовано на диаграмме модели угроз, — это то, что вы хотите защитить, потому что это может быть ступенькой к целям, которые хотят атакующие. В некотором смысле, набор активов-ступенек — это привлекательная неприятность. Например, каждый компьютер имеет CPU и хранилище, которые атакующий может использовать. У большинства также есть подключение к Интернету, и если вы в системном управлении или операционной безопасности, многие компьютеры, о которых вы больше всего беспокоитесь, будут иметь специальный доступ к сети вашей организации. Они за межсетевым экраном или имеют VPN-доступ. Это ступеньки. Если они уникально ценны как ступеньки в каком-то смысле, отметьте это. На практике редко бывает полезно включать «все наши ПК» в список активов.

ПРИМЕЧАНИЕ. Возвращаясь к примеру с сейфом в предыдущем разделе, комбинация сейфа — член семейства ступенек. Вполне возможно, что ступеньки и вещи, которые вы защищаете, на практике очень похожи. Список технических элементов, которые вы защищаете и которые не являются членами семейства ступенек, кажется довольно коротким.

Реализация моделирования, ориентированного на активы

Если бы вы моделировали угрозы с подходом, ориентированным на активы, вы бы составили список ваших активов, а затем рассмотрели, как атакующий мог бы угрожать каждому. Оттуда вы бы рассмотрели, как устранить каждую угрозу.

После создания списка активов вы должны связать каждый пункт списка с конкретными компьютерными системами или наборами систем. (Если актив включает что-то вроде «Amazon Web Services» или «Microsoft Azure», тогда вам не нужно указывать на рассматриваемые компьютерные системы, вам просто нужно понимать, где они находятся — в конечном итоге вам нужно будет определить угрозы для этих систем и определить, как их смягчить.)

Следующий шаг — нарисовать рассматриваемые системы, показывая активы и другие компоненты, а также взаимосвязи, пока вы не сможете рассказать о них историю. Вы можете использовать эту модель, чтобы применить либо набор атак, такой как STRIDE, либо мозговой штурм, ориентированный на атакующего, чтобы понять, как эти активы могут быть атакованы.

Перспектива на моделирование угроз, ориентированное на активы

Фокус на активах кажется подходом здравого смысла к моделированию угроз, до такой степени, что с ним трудно спорить. К сожалению, большую часть времени обсуждение активов не улучшает моделирование угроз. Однако это заблуждение настолько распространено, что важно рассмотреть, почему оно не помогает.

Нет прямой линии от активов к угрозам и нет предписывающего набора шагов. По сути, усилия, вложенные в перечисление активов, — это усилия, которые вы не тратите на

поиск или исправление угроз. Иногда это включает обсуждение того, что такое актив или какой тип актива вы обсуждаете. Это обсуждение в лучшем случае приводит к списку вещей, которые нужно искать в вашей модели ПО или операционной модели, так почему бы не начать с создания такой модели? Как только у вас есть список активов, этот список не является (кхм) ступенькой к поиску угроз; вам всё ещё нужно применить какую-то методологию или подход. Наконец, активы могут помочь вам приоритезировать угрозы, но если это ваша цель, это не значит, что вы должны начинать с активов или фокусироваться на них. Обычно такая информация естественно появляется при обсуждении воздействий, когда вы приоритезируете и устраняете угрозы. Эти темы рассматриваются в части III «Managing and Addressing Threats».

То, как вы отвечаете на вопрос «каковы наши активы?», должно помочь сфокусировать ваше моделирование угроз. Если это не помогает, нет смысла задавать вопрос или тратить время на ответ.

Фокус на атакующих

Фокус на атакующих кажется естественным способом моделировать угрозы. В конце концов, если никто не собирается атаковать вашу систему, зачем вам беспокоиться о её защите? И если вы беспокоитесь, потому что люди будут атаковать ваши системы, разве вы не должны их понимать? К сожалению, как и моделирование угроз, ориентированное на активы, моделирование угроз, ориентированное на атакующих, менее полезно, чем вы можете ожидать. Но есть также небольшое число сценариев, в которых фокус на атакующих может пригодиться, и это те же сценарии, что и с активами: эксперты, менее технический вклад в ваш процесс и приоритизация. И аналогично разделу «Фокус на активах», вы также можете узнать для себя, почему этот подход не оптимален, чтобы вы могли обсудить возможность с теми, кто его продвигает.

Реализация моделирования, ориентированного на атакующих

Эксперты по безопасности могут использовать различные типы списков атакующих для поиска угроз против системы. При этом легко обнаружить себя спорящим о ресурсах или возможностях такого архетипа и необходимости их конкретизации. Например, что, если ваш террорист спонсируется государством и имеет доступ к правительственным лабораториям? Эти вопросы делают подход, ориентированный на атакующих, похожим на «персоны», которые часто используются для помощи в размышлениях о проблемах человеческого интерфейса. Существует спектр детализации в моделях атакующих, от простых списков до персон, выведенных из данных, и примеры каждого приведены в Приложении C «Attacker Lists». Это приложение может помочь экспертам по безопасности и поможет любому, кто хочет попробовать моделирование, ориентированное на атакующих, и научиться быстрее, чем если бы ему пришлось начинать с создания списка.

Имея список атакующих, можно использовать его, чтобы обеспечить некоторую структуру для подхода мозгового штурма. Некоторые эксперты по безопасности используют списки атакующих как способ помочь выявить знания, накопленные ими по мере становления экспертами. Подходы, управляемые атакующими, также, вероятно, поднимут возможности, ориентированные на человека. Например, думая о том, что сделал бы шпион, может быть более естественно (и весело) думать о том, как он соблазняет вашего сисадмина или подкупает уборщика, а не думать о технических атаках. Хуже того,

вероятно, будет сложно подумать о том, что эти человеческие атаки означают для безопасности вашей системы.

Где атакующие могут помочь

Разговор о человеческих агентах угроз может помочь сделать угрозы реальными. То есть иногда трудно понять, как кто-то мог бы изменить конфигурационный файл или заменить клиентское ПО, чтобы обойти проверки безопасности. Особенно при работе с руководством или продуктовыми командами, которые «просто хотят выпустить», полезно уметь объяснить, кто может их атаковать и почему. В этом есть реальная ценность, но это не достаточный аргумент для центрирования вашего подхода на этих агентах угроз; вы можете добавить эту информацию на более позднем этапе. (Риск, связанный с разговором об атакующих, — это утверждение, что «никто никогда не стал бы этого делать». Попытка очеловечить риск, добавив действующее лицо, может усугубить это, особенно если вы добавляете тип действующего лица, которое, по мнению кого-то, «не заинтересовано в нас».)

Вам обещали пример, но шпионы его украли. Серьёзнее, тщательное прохождение по спискам атакующих и персонам в Приложении C, вероятно, не поможет вам (или автору) понять, что они могли бы захотеть сделать с Acme/SQL, поэтому пример оставлен пустым, чтобы избежать ложных надежд.

Перспектива на моделирование, ориентированное на атакующих

Помощь экспертам по безопасности в структурировании и воспоминании информации — это хорошо, но не приводит к воспроизводимым результатам. Что ещё важнее, списки атакующих или даже персоны — недостаточно структуры для большинства людей, чтобы понять, что эти люди будут делать. Инженеры могут подсознательно проецировать свои собственные предубеждения или подходы на то, что может сделать атакующий. Учитывая, что у атакующего есть свои собственные мотивации, навыки,背景 и перспектива (и, возможно, организационные приоритеты), избежать такой проекции сложно.

По моему опыту, эта комбинация проблем делает подходы, ориентированные на атакующих, менее эффективными, чем другие подходы. Поэтому я рекомендую против использования атакующих в качестве центра вашего процесса моделирования угроз.

Фокус на ПО

Хорошие новости! Вы официально достигли «лучшего» структурированного подхода к моделированию угроз. Поздравляю! Читайте дальше, чтобы узнать о моделировании угроз, ориентированном на ПО, почему это наиболее полезный и эффективный подход и как его делать.

Модели, ориентированные на ПО, — это модели, которые фокусируются на ПО, которое создаётся, или системе, которая развёртывается. Некоторые программные проекты имеют документированные модели различных видов, такие как архитектура, диаграммы UML или API. Другие проекты не утруждают себя такой документацией и вместо этого полагаются на неявные модели.

Рисование на доске большими проектными командами, чтобы объяснить, как ПО соединяется вместе, может быть на удивление полезной и спорной деятельностью.

Понимания различаются, особенно в больших проектах, которые работают давно, но нахождение того, где эти понимания различаются, может быть полезным само по себе, потому что это предлагает точку фокуса, где угрозы вряд ли будут заблокированы. («Я думал, ты проверяешь это для SQL-вызова!»)

Та же сложность применима к любому проекту, который больше нескольких человек или работает дольше нескольких лет. Проекты накапливают сложность, что делает многие аспекты разработки сложнее, включая безопасность. Моделирование угроз, ориентированное на ПО, может иметь полезный побочный эффект — выявление этой накопленной сложности.

Ценность безопасности этого общего понимания также может быть существенной, даже до того, как вы перейдёте к поиску угроз. В одном проекте оказалось, что у библиотеки на границе доверия была действительно хорошая модель угроз, но были сделаны нереалистичные предположения о том, что делают компоненты за ней. Работа по созданию всеобъемлющей модели привела к явному списку общих предположений, которые можно и нельзя делать. Всеобъемлющая модель и результирующее понимание привели к существенному улучшению безопасности этих компонентов.

ПРИМЕЧАНИЕ. По мере роста сложности будут расти и предположения, и такие списки никогда не бывают полными. Они растут по мере необходимости опыта и обратной связи.

Моделирование угроз для разных типов ПО

Подходы к обнаружению угроз, рассмотренные в части II, могут быть применены к моделям всех видов ПО. Они могут быть применены к ПО, которое вы создаёте для других, чтобы скачать и установить, а также к ПО, которое вы встраиваете в более крупную операционную систему. ПО, к которому они могут быть применены, почти бесконечно и не зависит от бизнес-модели или модели развёртывания, связанной с ПО.

Даже если ПО больше не поставляется в коробках, продаваемых на полках магазинов, термин «коробочное ПО» — удобная метка для категории. Эта категория — всё ПО, архитектура которого определима, потому что есть чёткая граница того, что является ПО: это всё в коробке (установщик, загрузка приложения или репозиторий с открытым исходным кодом). Эту границу можно противопоставить развёрнутым системам, которые организации разрабатывают и изменяют со временем.

ПРИМЕЧАНИЕ. Вас может беспокоить, что техники в этой книге сосредоточены либо на коробочном ПО, либо на развёрнутых системах, и что та, о которой вы беспокоитесь, не охвачена. В интересах экономии места примеры и обсуждение охватывают оба, только когда есть чёткая разница и причина. Это потому, что рекомендуемые способы моделирования ПО будут работать для обоих с лишь несколькими исключениями.

Граница между моделями коробочного ПО и сетевыми моделями становится всё более размытой с каждым годом. Одно важное различие в том, что сетевые модели, как правило, включают больше инфраструктурных компонентов, таких как маршрутизаторы, коммутаторы и каналы передачи данных. Границы доверия часто операционализируются этими компонентами или группой, которая управляет сетью, платформами или приложениями.

Модели потоков данных (с которыми вы познакомились в главе 1 «Dive In and Threat Model!» и о которых узнаете больше в следующем разделе) обычно являются хорошим выбором как для коробочного ПО, так и для операционных моделей. Некоторые крупные операторы центров обработки данных предоставили командам модели угроз, показывающие, как устроен центр обработки данных. Продуктовая группа может затем наложить свои модели «поверх» этого, чтобы согласовать с соответствующими средствами безопасности, которые они получат от эксплуатации. Когда вы используете чей-то чужой центр обработки данных, у вас могут быть обсуждения об их инфраструктурных решениях, которые позволяют легко вывести модель, или вам, возможно, придётся предполагать худшее.

Перспектива на моделирование, ориентированное на ПО

Я питаю слабость к подходам, ориентированным на ПО, потому что вы должны ожидать, что разработчики ПО понимают ПО, которое они разрабатывают. Действительно, нет ничего другого, что вы должны ожидать, что они поймут лучше. Это делает ПО идеальным местом для начала задач моделирования угроз, в которых вы просите разработчиков участвовать. Почти вся разработка ПО ведётся с использованием моделей ПО, которые достаточно хороши для целей команды. Иногда требуется работа, чтобы сделать их достаточно хорошими для эффективного моделирования угроз.

Напротив, вы можете лишь надеяться, что разработчики понимают бизнес или его активы. Вы можете стремиться к тому, чтобы они понимали людей, которые будут атаковать их продукт или систему. Но это надежды и стремления, а не разумные ожидания. В той мере, в какой ваша стратегия моделирования угроз зависит от этих надежд и стремлений, вы добавляете места, где она может потерпеть неудачу. Остальная часть этой главы посвящена моделированию вашего ПО способами, которые помогают находить угрозы, и, таким образом, обеспечивают моделирование, ориентированное на ПО. (Методы поиска этих угроз рассматриваются в остальной части части II.)

Модели ПО

Создание явной модели вашего ПО помогает вам искать угрозы, не увязая в множестве деталей, необходимых для правильного функционирования ПО. Диаграммы — естественный способ моделировать ПО.

Как вы узнали в главе 1, диаграммы на доске — чрезвычайно эффективный способ начать моделирование угроз, и они могут быть достаточными для вас. Однако, когда система достигает определённого уровня сложности, рисование и перерисовывание на досках становится неосуществимым. В этот момент вам нужно либо упростить систему, либо привлечь компьютеризированный подход.

В этом разделе вы узнаете о различных типах диаграмм, как их можно адаптировать для использования в моделировании угроз и как справляться со сложностями больших систем. Вы также узнаете больше деталей о границах доверия, эффективной маркировке и о том, как проверять ваши диаграммы.

Типы диаграмм

Есть много способов диаграммировать, и разные диаграммы помогут в разных обстоятельствах. Типы диаграмм, с которыми вы столкнётесь чаще всего, — вероятно, диаграммы потоков данных (DFD). Однако вы также можете увидеть UML, диаграммы swim lane и диаграммы состояний. Вы можете думать об этих диаграммах как о блоках Lego, просматривая их, чтобы увидеть, какой лучше подходит для того, что вы создаёте. Каждый тип диаграммы здесь может использоваться с моделями угроз из части II.

Цель всех этих диаграмм — сообщить, как работает система, чтобы у всех, участвующих в моделировании угроз, было одинаковое понимание. Если вы не можете договориться о том, как нарисовать, как работает ПО, то в процессе достижения согласия вы с высокой вероятностью обнаружите непонимание безопасности системы. Поэтому используйте тип диаграммы, который помогает вам вести хороший разговор и развивать общее понимание.

Диаграммы потоков данных

Модели потоков данных часто идеальны для моделирования угроз; проблемы, как правило, следуют за потоком данных, а не за потоком управления. Модели потоков данных чаще существуют для сетевых или архитектурных систем, чем для программных продуктов, но они могут быть созданы для любого из них.

Диаграммы потоков данных используются так часто, что их иногда называют «диаграммами модели угроз». Как изложил Ларри Константин в 1967 году, DFD состоят из пронумерованных элементов (хранилищ данных и процессов), соединённых потоками данных, взаимодействующих с внешними сущностями (теми, что вне контроля разработчика или организации).

Потоки данных, которые дают DFD их название, почти всегда текут в двух направлениях, с исключениями, такими как радиопередачи или UDP-данные, отправленные в Ethernet. Несмотря на это, потоки обычно представляются с помощью однонаправленных стрелок, поскольку угрозы и их воздействие, как правило, не симметричны. То есть, если данные, текущие к веб-серверу, читаются, это может раскрыть пароли или другие данные, тогда как поток данных от веб-сервера может раскрыть ваш банковский баланс. Это соглашение диаграммирования не помогает прояснить безопасность канала в сравнении с безопасностью сообщения. (Канал может быть чем-то вроде SMTP, а сообщения — электронными письмами.) Диаграммы swim lane могут быть более подходящими в качестве модели, если это различие канал/сообщение важно. (Диаграммы swim lane описаны в одноимённом подразделе позже в этой главе.)

Основные элементы диаграммы потоков данных показаны в таблице 2-1.

Таблица 2-1: Элементы диаграммы потоков данных

ЭЛЕМЕНТ	ВНЕШНИЙ ВИД	ЗНАЧЕНИЕ	ПРИМЕРЫ
Процесс	Скруглённый прямоугольник, круг	Любой выполняющийся код	Код, написанный на C, C#, Python или PHP

ЭЛЕМЕНТ	ВНЕШНИЙ ВИД	ЗНАЧЕНИЕ	ПРИМЕРЫ
	или концентрические круги		
Поток данных	Стрелка	Связь между процессами или между процессами и хранилищами данных	Сетевые соединения, HTTP, RPC, LPC
Хранилище данных	Две параллельные линии с меткой между ними	Вещи, которые хранят данные	Файлы, базы данных, реестр Windows, сегменты разделяемой памяти
Внешняя сущность	Прямоугольник с острыми углами	Люди или код вне вашего контроля	Ваш клиент, Microsoft.com

Рисунок 2-3 показывает классическую DFD, основанную на элементах из таблицы 2-1; однако можно сделать эти модели более удобными. Рисунок 2-4 показывает эту же модель с несколькими изменениями, которые вы можете использовать как пример для улучшения ваших собственных моделей.

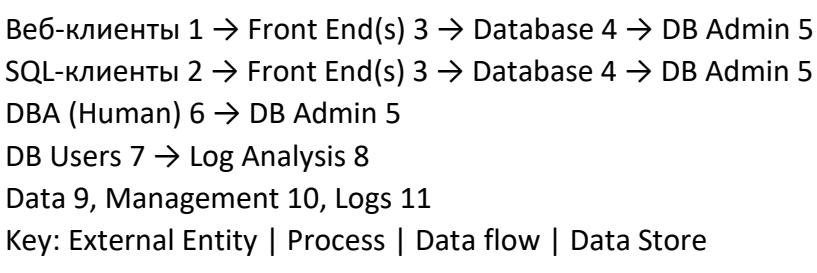


Рисунок 2-3: Классическая модель DFD

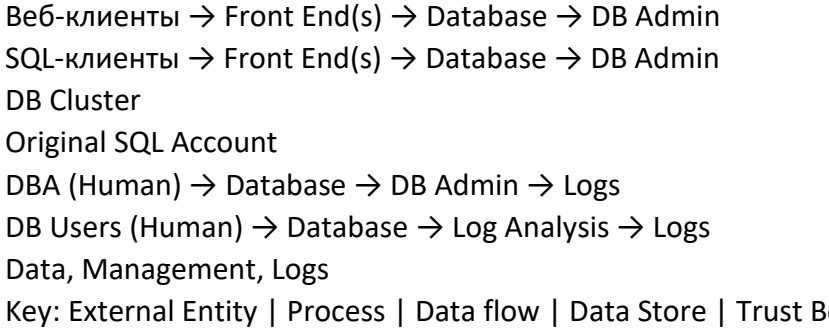


Рисунок 2-4: Современная модель DFD (ранее показанная как рисунок 2-1)

Следующий список объясняет изменения, сделанные от классических DFD к более современным:

- Процессы — это скруглённые прямоугольники, которые содержат текст более эффективно, чем круги. Используются прямые линии, а не кривые, потому что прямые линии легче проследить, и вы можете вместить больше в большие диаграммы.
- Исторически многие описания диаграмм потоков данных содержали как элементы «процесс», так и элементы «сложный процесс». Процесс изображался как круг, сложный процесс — как два концентрических круга. Не совсем ясно, однако, когда использовать обычный процесс, а когда сложный. Одно из возможных правил: всё, что имеет поддиаграмму, должно быть сложным процессом. Это кажется приличным правилом, если (кхм) немного круговым.

DFD могут использоваться для вещей, отличных от программных продуктов. Например, рисунок 2-5 показывает пример операционной сети в DFD. Это типичная модель для малой и средней корпоративной сети, с репрезентативной выборкой систем и отделов. Она подробно обсуждается в Приложении E «Case Studies».

Интернет → Desktop & Mobile → E-mail & Intranet Servers → Development Servers
Production → Directory → HR Mgmt
Operations → Sales/CRM → Payroll
Acme Corporate Network

Рисунок 2-5: Модель операционной сети

UML

UML — это аббревиатура для Unified Modeling Language. Если вы используете UML в вашем процессе разработки ПО, вполне вероятно, что вы можете адаптировать диаграммы UML для моделирования угроз, а не перерисовывать их. Наиболее важный способ адаптации UML для диаграмм моделирования угроз — добавление границ доверия.

UML довольно сложен. Например, трафареты Visio для UML предлагают примерно 80 символов по сравнению с шестью для DFD. Эта сложность приносит изрядную долю нюансов и выразительности, когда люди рисуют структурные диаграммы, диаграммы поведения и диаграммы взаимодействия. Если кто-либо из участвующих в моделировании угроз не знает всех символов UML или если есть непонимание того, что эти символы означают, эффективность диаграммы как инструмента значительно снижается. Теоретически любой, кто запутался, может просто спросить, но для этого ему нужно знать, что он запутался (он может предположить, что символ рыбы исключает акул). Это также требует готовности выставить своё невежество, задав «простой» вопрос. Вероятно, команде, которая инвестирована в UML, легче добавить границы доверия к этим диаграммам, чем создавать новые диаграммы только для моделирования угроз.

Диаграммы swim lane

Диаграммы swim lane — распространённый способ представления потоков между различными участниками. Они рисуются с использованием длинных линий, каждая из которых представляет участников протокола, причём каждый участник получает линию. Край каждой дорожки помечен для идентификации участника; каждое сообщение

представлено линией между участниками; время представлено потоком вниз по дорожкам диаграммы. Диаграммы в итоге выглядят немного как дорожки бассейна, отсюда и название. Сообщения должны быть помечены их содержимым; или, если содержимое сложное, может иметь больше смысла иметь ключ диаграммы, который абстрагирует некоторые детали. Вычисления, выполняемые сторонами или состоянием, должны быть отмечены вдоль линии этого участника. Обычно участники в таких протоколах — это сущности вроде компьютеров; и как таковые, диаграммы swim lane обычно имеют неявные границы доверия между каждым участником. Криптограф и проектировщик протоколов Карл Эллисон расширил swim lanes, включив человеческих участников как способ структурировать обсуждение того, что люди, как ожидается, должны знать и делать. Он называет это расширение «церемониями», которое более подробно обсуждается в главе 15 «Human Factors and Usability».

Пример диаграммы swim lane показан на рисунке 2-6.

Клиент → SYN → Сервер
Сервер → SYN-ACK → Клиент
Клиент → ACK → Сервер
Клиент → Data → Сервер

Рисунок 2-6: Диаграмма swim lane (показывающая начало TCP-соединения)

Диаграммы состояний

Диаграммы состояний представляют различные состояния, в которых может находиться система, и переходы между этими состояниями. Компьютерная система моделируется как машина с состоянием, памятью и правилами перехода из одного состояния в другое на основе допустимых сообщений, которые она получает, и данных в её памяти. (Компьютер должен, конечно, проверять получаемые сообщения на валидность согласно некоторым правилам.) Каждый блок помечен состоянием, а линии между ними помечены условиями, вызывающими переход состояния. Вы можете использовать диаграммы состояний в моделировании угроз, проверяя, управляется ли каждый переход в соответствии с соответствующими проверками безопасности.

Очень простая машина состояний для двери показана на рисунке 2-7 (производная от Wikipedia). Дверь имеет три состояния: opened, closed и locked. Каждое состояние вводится переходом. Система «задвижки» гораздо проще для рисования, чем замки на ручке, которые можно запереть из любого состояния, создавая более сложную диаграмму и пользовательский опыт. Очевидно, диаграммы состояний могут быстро становиться сложными. Вы могли бы представить более сложную диаграмму состояний, которая включает «приоткрыто», состояние, которое может возникнуть из open или closed. (Я начал рисовать это, но у меня возникли проблемы с решением о метках. Очевидно, двери, которые могут быть приоткрыты, плохо специфицированы и не должны развёртываться.) Вы не хотите принимать архитектурные решения только для того, чтобы упростить моделирование, но часто простые модели легче в работе и отражают лучшую инженерию.

Открыто → Close door → Закрыто
Закрыто → Open door → Открыто

Закрыто → Lock deadbolt → Заперто
Заперто → Unlock deadbolt → Закрыто
Transition condition → Transition → State

Рисунок 2-7: Диаграмма машины состояний

Границы доверия

Как вы видели в главе 1, граница доверия — это любое место, где различные принципалы сходятся вместе — то есть где взаимодействуют сущности с разными привилегиями.

Рисование границ

После того как модель ПО нарисована, есть два способа добавить границы: вы можете добавить известные вам границы и искать更多, или вы можете перечислить принципалов и искать границы. Чтобы начать с границ, добавьте любые виды принудительных границ доверия, которые сможете. Границы между UID Unix, сессиями Windows, машинами, сетевыми сегментами и так далее должны быть нарисованы как рамки, и принципал внутри каждой рамки должен быть показан с меткой.

Чтобы начать с принципалов, начните с одного конца спектра привилегий (часто это root/admin или анонимные интернет-пользователи), а затем добавляйте границы каждый раз, когда они говорят с «кем-то другим».

Вы всегда можете добавить как минимум одну границу, так как все вычисления происходят в каком-то контексте. (Так что вы могли бы критиковать рисунок 2-1 за показ Web Clients и SQL Clients без идентифицированного контекста.)

Если вы не видите, где рисовать границы доверия любого рода, ваша диаграмма может быть детализирована настолько, что всё находится внутри одной границы доверия, или вы можете пропускать границы. Задайте себе два вопроса. Во-первых, имеет ли всё в системе одинаковый уровень привилегий и доступ ко всему остальному в системе? Во-вторых, всё, с чем взаимодействует ваше ПО, находится внутри той же границы? Если ответ на любой из них — нет, то теперь вы должны были прояснить либо пропущенную границу, либо пропущенный элемент диаграммы, либо и то, и другое. Если оба — да, то вы должны нарисовать одну границу доверия вокруг всего и перейти к другим задачам разработки. (Это состояние маловероятно, за исключением случаев, когда каждая часть команды разработчиков должна создать модель ПО. Этот «восходящий» подход более подробно обсуждается в главе 7 «Processing and Managing Threats».)

Много литературы по моделированию угроз утверждает, что границы доверия должны пересекать только потоки данных. Это полезный совет для самого детального уровня вашей модели. Если граница доверия пересекает хранилище данных (то есть базу данных), это может указывать на то, что есть разные таблицы или хранимые процедуры с разными уровнями доверия. Если граница пересекает данный хост, это может отражать, что члены, например, группы «установщики ПО» имеют разные права от «обновляющих веб-контент». Если вы найдёте границу доверия, пересекающую элемент диаграммы, отличный от потока данных, либо разбейте этот элемент на два (в модели, в реальности или и то, и другое), либо нарисуйте поддиаграмму, чтобы показать их разделёнными на несколько сущностей. Что обеспечивает хорошие модели угроз — это ясность о том, какие

границы существуют и как эти границы должны быть защищены. Напротив, отсутствие ясности будет препятствовать созданию хороших моделей.

Использование границ

Угрозы, как правило, кластеризуются вокруг границ доверия. Это может показаться очевидным: границы доверия очерчивают поверхность атаки между принципалами. Это заставляет некоторых ожидать, что угрозы появляются только между принципалами на границе или важны только на границах доверия. Это ожидание иногда неверно. Чтобы увидеть почему, рассмотрите веб-сервер, выполняющий некоторую сложную обработку заказов. Например, представьте сборку компьютера в интернет-магазине Dell, где можно добавить тысячи деталей, но только подмножество из них протестировано и предлагается. Модель этого веб-сайта могла бы быть построена, как показано на рисунке 2-8.

Order processing → Sales ↔ Marketing
Application
Web server
Platform
Web browser ↔ TCP/IP stack
Kernel

Рисунок 2-8: Границы доверия в веб-сервере

Веб-сервер на рисунке 2-8 явно подвержен риску атаки со стороны веб-браузера, даже если он общается через стек TCP/IP, которому он, предположительно, доверяет. Аналогично, модуль Sales подвержен риску; плюс атакующий мог бы вставить случайные номера деталей в HTML-пост, в котором данные проверяются в модуле обработки заказов. Даже если нет границы доверия между модулем Sales и модулем Order processing, и даже если данные могут проверяться на трёх границах, угрозы всё равно следуют за потоками данных. Клиент показан просто как веб-браузер, потому что клиент — это внешняя сущность. Конечно, вокруг этого веб-браузера есть много других компонентов, но вы ничего не можете сделать с угрозами для них, так зачем их моделировать?

Поэтому более точно сказать, что угрозы, как правило, кластеризуются вокруг границ доверия и сложного парсинга, но могут появляться везде, где информация находится под контролем атакующего.

Что включать в диаграмму

Итак, что должно быть в вашей диаграмме? Некоторые правила:

- Покажите события, которые движут системой.
- Покажите процессы, которые движутся.
- Определите, какие ответы каждый процесс будет генерировать и отправлять.
- Определите источники данных для каждого запроса и ответа.
- Определите получателя каждого ответа.

- Игнорируйте внутреннюю работу, сосредоточьтесь на области охвата.
- Спросите, поможет ли что-то вам подумать о том, что идёт не так, или что поможет вам найти угрозы.

Этот список derived from Howard and LeBlanc's Writing Secure Code, Second Edition (Microsoft Press, 2009).

Сложные диаграммы

Когда вы строите сложные системы, вы можете получить сложные диаграммы. Системы действительно становятся сложными, и эта сложность может затруднить использование диаграмм (или понимание всей системы).

Одно правило: «не рисуйте глазную диаграмму». Важно сбалансировать все детали, которые может повлечь реальный программный проект, с тем, что вы включаете в свою фактическую модель. Как упоминалось в главе 1, одна из техник, которую вы можете использовать, чтобы помочь вам сделать это, — это поддиаграмма, показывающая детали одной конкретной области. Вам следует искать способы выделить высокодетализированные области, которые имеют смысл для вашего проекта. Например, если у вас есть один очень сложный процесс, может быть, всё внутри него — одна диаграмма, а всё снаружи — другая. Если у вас есть диспетчер или система очередей, это может быть хорошим местом для разбиения. Может быть, ваши базы данных или система отказоустойчивости — хорошее место для разделения. Может быть, есть несколько элементов, которые действительно нуждаются в большей детализации. Всё это — хорошие способы выделить детали.

Один полезный подход к поддиаграммам — убедиться, что поддиаграмм не больше, чем процессов. Другой подход — использовать разные диаграммы для показа разных сценариев.

Иногда также полезно упрощать диаграммы. Когда два элемента диаграммы эквивалентны с точки зрения безопасности, вы можете объединить их. Эквивалентность означает внутри одной границы доверия, полагаясь на одну и ту же технологию и обрабатывая один и тот же тип данных.

Главное помнить: диаграмма предназначена, чтобы помочь вам понять и обсудить систему. Вспомните цитату, открывающую книгу: «Все модели неверны. Некоторые модели полезны». Поэтому, когда вы добавляете дополнительные диаграммы, не спрашивайте: «Это правильный способ делать это?» Вместо этого спросите: «Это помогает нам подумать о том, что может пойти не так?»

Метки на диаграммах

Метки на диаграммах должны быть короткими, описательными и осмысленными. Поскольку вы хотите использовать эти имена для рассказывания историй, начните с посторонних, которые движут системой; это существительные, такие как «клиент» или «датчик вибрации». Они передают информацию через потоки данных, которые являются существительными или именными фразами, такими как «книги для покупки» или «частота вибрации». Потоки данных почти никогда не должны быть помечены с помощью глаголов. Хотя это может быть трудно, вы должны работать, чтобы найти более

описательные метки, чем «читать» или «писать», которые подразумеваются направлением стрелок. Другими словами, потоки данных передают свою информацию (существительные) процессам, которые активны: глаголы, глагольные фразы или цепочки глагол/существительное.

Многие люди находят полезным помечать потоки данных порядковыми номерами, чтобы помочь отслеживать, что происходит в каком порядке. Также может быть полезно нумеровать элементы внутри диаграммы, чтобы помочь с полнотой или коммуникацией. Вы можете нумеровать каждую вещь (поток данных 1, процесс 1 и т.д.) или у вас может быть единый счёт по диаграмме, с внешней сущностью 1, говорящей через потоки данных 2 и 3 к процессу 4. Обычно использование единого счётчика для всего менее запутанно. Вы можете сказать «номер 1», а не «поток данных 1, не процесс 1».

Цвет на диаграммах

Цвет может добавить существенное количество информации, не выглядя подавляющим. Например, Питер Торр из Microsoft использует зелёный для доверенного, красный для недоверенного и синий для того, что моделируется (Torr, 2005). Полагаться только на цвет может быть проблематично. Примерно один из двенадцати человек страдает дальтонизмом, наиболее распространённым является красно-зелёная путаница (Heitgerd, 2008). Результат в том, что даже с цветным принтером существенное число людей не могут легко получить доступ к этой критической информации. Границы-рамки с текстовыми метками решают обе проблемы. С границами доверия в виде рамок нет причин не использовать цвет.

Точки входа

Один из ранних подходов к моделированию угроз был подход «актив/точка входа», который может быть эффективен при моделировании операционных систем. Этот подход можно частично разбить на следующие шаги:

1. Нарисуйте DFD.
2. Найдите точки, где потоки данных пересекают границы доверия.
3. Помечьте эти пересечения как «точки входа».

ПРИМЕЧАНИЕ. Были другие шаги и variations в подходах, но мы как сообщество узнали много с тех пор, и полное объяснение было бы утомительным и отвлекающим.

В примере Acme/SQL (как показано на рисунке 2-1) точки входа — это «front end(s)» и консольный процесс «database admin». «Database» также была бы точкой входа, потому что номинально другое ПО могло бы изменять данные в базах данных и использовать сбои в парсерах для получения контроля над системой. Для финансовых отчётов показанные точки входа — это «external reporting», «financial planning and analysis», «core finance software», «sales» и «accounts receivable».

Валидация диаграмм

Валидация того, что диаграмма является хорошей моделью вашего ПО, имеет две основные цели: обеспечение точности и стремление к хорошести. Первое легче, так как вы можете спросить, отражает ли она реальность. Если важные компоненты отсутствуют

или диаграмма показывает вещи, которые не строятся, то вы можете видеть, что она не отражает реальность. Если важные потоки данных отсутствуют или показаны несуществующие потоки, то она не отражает реальность. Если вы не можете рассказать историю о ПО без редактирования диаграммы, то она неточна.

Конечно, есть это слово «важные», которое ведёт ко второму критерию: стремление к хорошести. Важно то, что помогает вам находить проблемы. Нахождение проблем — это вопрос задавания вопросов вроде «имеет ли этот элемент какое-либо влияние на безопасность?» и «есть ли вещи, которые происходят иногда или в особых обстоятельствах?» Знание ответов на эти вопросы — это вопрос опыта, как и многие аспекты создания ПО. Хороший и опытный архитектор может быстро оценить требования и address их, и хороший специалист по моделированию угроз может быстро увидеть, какие элементы будут важны. Большая часть получения этого опыта — практика. Структурированные подходы к поиску угроз в части II предназначены, чтобы помочь вам определить, какие элементы важны.

Как валидировать диаграммы

Чтобы лучше всего валидировать ваши диаграммы, соберите людей, которые лучше всего понимают систему. Кто-то должен встать перед диаграммой и пройти через важные варианты использования, убедившись в следующем:

- Они могут рассказать истории о диаграмме.
- Им не нужно вносить изменения в диаграмму в её текущем виде.
- Им не нужно ссылаться на вещи, отсутствующие на диаграмме.

Следующие правила будут полезны при обновлении диаграммы и накоплении опыта:

- Каждый раз, когда вы обнаружите, что кто-то говорит «иногда» или «также», вам следует рассмотреть добавление更多 деталей, чтобы разбить различные случаи. Например, если вы говорите: «Иногда мы подключаемся к этому веб-сервису через SSL, а иногда откатываемся к HTTP», вам следует нарисовать оба этих потока данных (и рассмотреть, может ли атакующий заставить вас так откатиться).
- Каждый раз, когда вам нужно больше деталей, чтобы объяснить поведение, относящееся к безопасности, рисуйте их.
- Каждая рамка границы доверия должна иметь метку внутри неё.
- В любом месте, где вы спорили о дизайне или конструкции системы, впишите эти детали. Это важный шаг, чтобы убедиться, что все закончили обсуждение на одной странице. Это особенно важно для больших команд, когда не все находятся в комнате для обсуждения модели угроз. Если кто-то видит диаграмму, которая противоречит его мышлению, он может либо принять её, либо оспорить предположения; но в любом случае хорошая чёткая диаграмма может помочь всем прийти к общему пониманию.
- Не имейте стоков данных: вы пишете данные по какой-то причине. Покажите, кто их использует.

- Данные не могут перемещаться сами из одного хранилища в другое: покажите процесс, который их перемещает.
- Все способы, которыми данные могут прибыть, должны быть показаны.
- Если есть механизмы для контроля потоков данных (такие как межсетевые экраны или разрешения), они должны быть показаны.
- Все процессы должны иметь как минимум один входной поток данных и один выходной поток данных.
- Как обсуждалось ранее в главе, не рисуйте глазную диаграмму. Диаграммы должны быть видимы на печатной странице.

ПРИМЕЧАНИЕ. Автор Writing Secure Code Дэвид ЛеБлан отмечает: «Процесс без входа — это чудо, а без выхода — чёрная дыра. Либо вы что-то упускаете, либо ошибочно приняли процесс за людей, которым позволено быть чёрными дырами или чудесами».

Когда валидировать диаграммы

Для программных продуктов есть два основных времени для валидации диаграмм: когда вы их создаёте и когда готовитесь выпустить бета-версию. Есть также третье триггерное событие (которое менее часто), — если вы добавляете границу безопасности.

Для диаграмм операционного ПО вы также валидируете, когда создаёте их, а затем снова, используя разумный баланс между усилиями и актуальностью. Этот разумный баланс будет варьироваться в зависимости от зрелости системы, её масштаба, того, насколько тесно связаны компоненты, частоты развёртываний и характера новых развёртываний. Вот несколько рекомендаций:

- Новые системы будут испытывать больше изменений диаграмм, чем зрелые.
- Большие системы будут испытывать больше изменений диаграмм, чем меньшие.
- Тесно связанные системы будут испытывать больше изменений диаграмм, чем слабо связанные.
- Системы, которые развёртывают изменения быстро, вероятно, будут испытывать меньше изменений диаграмм на одно развёртывание.
- Развёртывания или спринты, сосредоточенные на рефакторинге или погашении технического долга, вероятно, увидят больше изменений диаграмм. В любом случае создайте соответствующий элемент отслеживания, чтобы убедиться, что вы перепроверите свои диаграммы в подходящее время. Соответствующий элемент отслеживания — это то, что вы используете для гейтирования релизов или развёртываний, такое как баги, ПО управления задачами или контрольные списки. Если у вас нет формального способа гейтировать релизы, то вам, возможно, следует сосредоточиться на чётко определённом процессе релиза, прежде чем беспокоиться о перепроверке моделей угроз. Описание такого процесса выходит за рамки этой книги.

Резюме

Есть более одного способа моделировать угрозы, и некоторые из стратегий, которые вы можете использовать, включают моделирование активов, моделирование атакующих или моделирование ПО. «Какова ваша модель угроз» и мозговой штурм хороши для экспертов по безопасности, но им не хватает структуры, которая нужна менее опытным специалистам по моделированию угроз. Есть более структурированные подходы к мозговому штурму, включая анализ сценариев, пре-мортемы, киносюжеты и обзоры литературы, которые могут помочь привнести немного структуры, но они всё ещё не отличны.

Если ваше моделирование угроз начинается с активов, множественные пересекающиеся определения термина, включая вещи, которые хотят атакующие, вещи, которые вы защищаете, и ступеньки, могут сбить вас с толку. Подход, ориентированный на активы, не предлагает пути к выяснению того, что пойдёт не так с активами.

Моделирование атакующих также привлекательно, но попытка предсказать, как будет атаковать другой человек, сложна, и подход может приглашать споры о том, что «никто никогда не стал бы этого делать». Кроме того, подходы, ориентированные на человека, могут привести вас к человеко-ориентированным угрозам, которые трудно устранить.

Модели ПО сосредоточены на том, что понимают люди, работающие с ПО. Лучшие модели — это диаграммы, которые помогают участникам понять ПО и найти угрозы против него. Есть множество способов диаграммировать ваше ПО, и DFD наиболее часто полезны.

Как только у вас есть модель ПО, вам понадобится способ находить угрозы против неё, и это предмет части II.